

MAD Lab — Quick Exam Notes

Mobile Application Development Lab
Final Year CSE | Textile & Engineering Institute, Ichalkaranji

All 13 Experiments · Android (E01–E09) + iOS/Swift (E10–E13)

ANDROID SECTION (E01 – E09)

E01 — Android SDK, Emulator & Project Structure

What is Android?

- Linux-based mobile OS, originally by Android Inc., acquired by Google in 2005.
- Released under open-source Apache License — manufacturers can customize it.
- Goal: unified development platform. One codebase runs on many devices.

Tools Required:

- Java JDK 8 or later
- Android Studio (IDE)
- AVD Manager — creates Android Virtual Devices (emulators)

Android Project Structure:

- **res/layout/** — XML files defining UI screens
- **java/** — Java/Kotlin source code
- **AndroidManifest.xml** — declares all components, permissions, app metadata
- **Gradle** — build system; manages dependencies
- **res/values/strings.xml** — string resources (good practice to avoid hardcoding)

Viva tip: "Manifest is the app's blueprint. Every Activity, Service, and permission must be declared there."

E02 — Layouts in Android

Hierarchy: View → ViewGroup → Layout

- **View** = base class of all UI elements (occupies rectangle on screen)
- **ViewGroup** = invisible container that holds Views
- **Layout** = subclass of ViewGroup that arranges children

Layout Types:

- **LinearLayout** — arranges children in one direction (horizontal OR vertical)
- **RelativeLayout** — positions relative to parent or sibling views
- **ConstraintLayout** — most flexible; uses constraints between elements
- **FrameLayout** — stacks views on top of each other (like layers)
- **TableLayout** — grid structure using rows and columns

Two ways to declare layout:

1. XML file in **res/layout/** (preferred)
2. Programmatically at runtime using View/ViewGroup objects

Viva tip: "XML is preferred because it separates UI from logic, making code cleaner and easier to maintain."

E03 — Intents & Switching Between Activities

What is an Intent?

A messaging object used to request an action from another app component.

3 Use Cases:

1. **Start an Activity** — new screen/page in the app
2. **Start a Service** — background operation (no UI)
3. **Deliver a Broadcast** — system-wide message (e.g., battery low)

Types of Intents:

- **Explicit Intent** — you specify EXACT class to start. Used within your own app.
- **Implicit Intent** — system finds the right app. E.g., open browser, camera, maps.

Key Methods:

- **startActivity(intent)** — start activity, no result needed
- **startActivityForResult(intent, code)** — start and get result back
- **onActivityResult()** — callback to receive the result

Viva tip: "Explicit = I know exactly who I want. Implicit = I just describe what I want, system decides."

E04 — UI Widgets (Button, TextField, Checkbox, RadioButton, ToggleButton)

All widgets in package: android.widget

3 View Groups:

- **Basic Views** — TextView, EditText, Button (most common)
- **Picker Views** — TimePicker, DatePicker (select from options)
- **List Views** — ListView, Spinner (display lists)

Key Widgets:

- **TextView** — display text (read-only)
- **EditText** — user input text field
- **Button** — clickable button; handler: setOnClickListener()
- **CheckBox** — multi-select; handler: setOnCheckedChangeListener()
- **RadioButton** — single select; MUST be inside RadioGroup
- **ToggleButton** — on/off switch with ON/OFF label

Viva tip: "RadioButtons must go inside RadioGroup — otherwise multiple can be selected simultaneously."

E05 — Intents for SMS and Telephony

Telephony:

- **ACTION_DIAL** — opens dialer with pre-filled number. No permission needed.
- **ACTION_CALL** — directly calls. Requires CALL_PHONE permission in Manifest.

SMS:

- Use Intent.ACTION_SENDTO with URI: smsto:
- Attach message with: putExtra("sms_body", message)

Runtime Permissions (Android 6.0+):

- Must request CALL_PHONE permission at runtime, not just in Manifest
- Use ActivityCompat.requestPermissions() and handle result

Viva tip: "ACTION_DIAL is safer — no permission. ACTION_CALL needs runtime permission from Android 6+."

E06 — Spinners, Alerts, Popups, Toasts

Spinner (Dropdown):

- Displays one item, touch shows dropdown list
- Populated via **ArrayAdapter** (from array) or **CursorAdapter** (from DB)
- Handler: setOnItemSelectedListener()

Toast:

- Short non-interactive message at bottom of screen
- Toast.makeText(context, "message", Toast.LENGTH_SHORT).show()

AlertDialog:

- Dialog box with title, message, up to 3 buttons (positive/negative/neutral)
- Built using AlertDialog.Builder

Touch Mode:

- Device enters touch mode when user first touches screen
- In touch mode, keyboard-focused highlights disappear

Viva tip: "Toast is fire-and-forget. AlertDialog waits for user response."

E07 — Touch Mode & Menus (Options, Context, Popup)

3 Types of Menus:

- **Options Menu** — in app bar (3-dot). Global app actions.
Methods: `onOptionsItemSelected()` + `onOptionsItemSelected()`
- **Context Menu** — appears on long-press of a view. View-specific actions.
Methods: `registerForContextMenu(view)` + `onContextItemSelected()`
- **Popup Menu** — small floating menu anchored to a specific view.
Use: `PopupMenu` class, inflate a menu resource

Touch Gestures:

- Capture via `onTouchListener()` or `GestureDetector` class

Viva tip: "Options = global (whole app). Context = local (one view). Popup = anchored (to a specific element)."

E08 — Notifications with Actions

What is a Notification?

Message in device status bar/notification drawer to alert user about an event.

Core Components:

- **Title + Content** — main text shown
- **Small Icon** — shown in status bar (mandatory)
- **Channel ID** — MANDATORY for Android 8.0+ (API 26). Groups notifications.
- **PendingIntent** — action that executes when user taps notification
- **NotificationManager** — posts notification to the system

How to build:

1. Create `NotificationChannel` (for Android 8+)
2. Build with `NotificationCompat.Builder`
3. Set `.setContentIntent(pendingIntent)` — for tap action
4. Add `.addAction()` — for buttons in notification
5. Call `notificationManager.notify(id, notification)`

Viva tip: "PendingIntent is a promise — it says 'when user taps, do this on behalf of my app.'"

E09 — SQLite Database

What is SQLite in Android?

- Built-in relational database. No external server or JDBC/ODBC needed.
- Private to app — stored at: `/data/data/<package_name>/databases/`

Key Class: `SQLiteOpenHelper`

- Override `onCreate()` — called when DB created first time
- Override `onUpgrade()` — called when DB version changes

CRUD Operations:

- **Create DB:** `openOrCreateDatabase("name", MODE_PRIVATE, null)`
- **INSERT:** `db.insert(tableName, null, contentValues)`
- **SELECT:** `db.rawQuery("SELECT * FROM table", null)` — returns `Cursor`
- **UPDATE:** `db.update(table, values, whereClause, whereArgs)`
- **DELETE:** `db.delete(table, whereClause, whereArgs)`

Cursor:

- Object used to iterate through query results (like `ResultSet` in JDBC)
- `moveToFirst()`, `moveToNext()`, `getString(colIndex)`, `getInt(colIndex)`

Viva tip: "SQLiteOpenHelper is the manager. Cursor is the iterator. ContentValues is the data container."

iOS / SWIFT SECTION (E10 – E13)

E10 — Xcode, iOS Simulator & Interface Builder

Xcode:

- Apple's official IDE for iOS, macOS, watchOS, tvOS development
- Includes: Interface Builder, iOS Simulator, Instruments (profiler)

iOS Simulator:

- Test apps on virtual iPhones/iPads — no physical device needed
- Simulates orientation, gestures, location, and more

Interface Builder:

- Drag-and-drop visual UI designer built into Xcode
- Saves UI in .storyboard or .xib files
- Link UI elements to code via Ctrl+Drag

Steps to create a project:

Xcode → New Project → App (iOS) → Enter name → Select Swift + Storyboard → Finish

Viva tip: "Storyboard = visual flow of screens. XIB = single view file. Storyboard is used for multi-screen apps."

E11 — UI Outlets, Buttons, Views & ImageViews in Xcode

Two Connection Types:

- **@IBOutlet** — creates reference to UI element in code (to change properties programmatically)
- **@IBAction** — handles user interaction (button tap, slider change, etc.)

Key UI Elements:

- **UILabel** — displays static/dynamic text
- **UIButton** — tappable action button
- **UIView** — general container; can be styled with color, border, corner radius
- **UIImageView** — displays an image (from assets or URL)

How to connect:

Open Assistant Editor → Ctrl+Drag from Storyboard element → drop into Swift file

Viva tip: "@IBOutlet = 'I can see and change this element.' @IBAction = 'I respond when user interacts with it.'"

E12 — Intermediate iOS UI (Constraints, View Controllers, MVC)

4 Key Concepts:

1. **@IBOutlet** — connect UI element to code for dynamic control
2. **Auto Layout Constraints** — define relationships between UI elements
 - Leading, trailing, top, bottom, width, height, center alignment
 - Ensures layout adapts to ALL screen sizes and orientations
3. **View Controller** — bridge between UI and data/logic
 - Lifecycle: viewDidLoad() → viewWillAppear() → viewDidAppear()
 - Manages what user sees and how app reacts to events
4. **MVC Pattern (Model-View-Controller):**
 - **Model** = data and business logic
 - **View** = UI (Storyboard elements)
 - **Controller** = ViewController.swift (connects model and view)

Viva tip: "Auto Layout replaces hardcoded coordinates. Constraints make UI responsive across iPhone SE to iPad Pro."

E13 — DatePicker, UIPickerView & Sliders in iOS

1. UIDatePicker:

- Select date and/or time
- Modes: .date, .time, .dateAndTime, .countDownTimer
- Get value: datePicker.date → format using DateFormatter

2. UIPickerView (Scrollable Wheel):

- Shows list of selectable items in wheel format
- Needs 2 delegate protocols:
 - **UIPickerViewDataSource** — numberOfComponents(), numberOfRows()
 - **UIPickerViewDelegate** — titleForRow(), didSelectRow()

3. UISlider:

- Select value from continuous range (e.g., 0 to 100)
- Event: .valueChanged — fires as user drags
- Get value: slider.value (Float)

Viva tip: "PickerView needs DataSource AND Delegate — both must be set. Forgetting either crashes the app."

VIVA CHEAT SHEET — All 13 Experiments at a Glance

Exp	Title	One-line viva answer
E01	Android Setup	Android is Linux-based OS. SDK + AVD installed. Project has Manifest, res/, java/.
E02	Layouts	View→ViewGroup→Layout. LinearLayout, RelativeLayout, ConstraintLayout used.
E03	Intents	Intent = messaging object. Explicit = known target. Implicit = system decides.
E04	UI Widgets	Basic widgets in android.widget. RadioButton must be in RadioGroup.
E05	SMS & Telephony	ACTION_DIAL = no permission. ACTION_CALL = CALL_PHONE permission needed.
E06	Spinners/Toasts	Spinner = dropdown via ArrayAdapter. Toast = quick msg. AlertDialog = blocking.
E07	Menus	Options=global, Context=long-press, Popup=anchored. Each has its own callbacks.
E08	Notifications	PendingIntent defines tap action. Channel ID mandatory for Android 8+.
E09	SQLite	SQLiteOpenHelper manages DB. Cursor iterates results. CRUD via db methods.
E10	Xcode Intro	Xcode IDE, Interface Builder for drag-drop UI, Simulator for testing without device.
E11	iOS Outlets	@IBOutlet = reference. @IBAction = event handler. Ctrl+Drag to connect.
E12	iOS Intermediate	MVC pattern. ViewController bridges UI and logic. Constraints = responsive layout.
E13	iOS Pickers	UIDatePicker for date/time. UIPickerView needs DataSource+Delegate. UISlider is continuous.

Good luck on your oral! — MAD Lab Notes generated for exam prep